

FOOD ORDERING SYSTEM

WEEK 3 REPORT

Seller and Inventory Management System

Introduction

The Food Ordering System is a database-based application used to manage food products, sellers, and inventory information. In Week 3, the main focus is on Seller and Inventory Management. The Seller table stores details of food suppliers, while the Inventory table stores product stock, stock status, and last updated date. The tables are connected using primary and foreign key constraints to maintain data integrity.

Objectives

- To create a Seller table for storing seller information.
- To create an Inventory table for storing food product stock information.
- To connect Inventory with the Product table created in Week 2.
- To connect Inventory with the Seller table.
- To maintain unique seller email addresses.
- To prevent negative stock quantities using a CHECK constraint.
- To insert and retrieve seller and inventory records.
- To identify available and out-of-stock products.
- To find the product with the maximum stock quantity.
- To calculate the average inventory quantity.

Database Tables

Seller Table

The Seller table stores information about sellers or suppliers who provide food products.

Column	Data Type	Description
SELLER_ID	VARCHAR(7)	Unique ID of seller
SELLER_NAME	VARCHAR(25)	Name of seller
EMAIL	VARCHAR(30)	Seller email address
PHONE	NUMBER(10)	Seller contact number
ADDRESS	VARCHAR(45)	Seller address

Inventory Table

The Inventory table stores the stock details of food products supplied by sellers.

Column	Data Type	Description
INVENTORY_ID	VARCHAR(7)	Unique inventory ID
PRODUCT_ID	VARCHAR(7)	ID of food product
SELLER_ID	VARCHAR(7)	ID of seller
STOCK_QUANTITY	NUMBER	Available stock quantity
STOCK_STATUS	VARCHAR(11)	Available or Out of Stock
LAST_UPDATED	DATE	Date of last stock update

Create Seller Table

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7), Seller_Name VARCHAR(25),Email VARCHAR(30),Phone NUMBER(10),Address VARCHAR(45));
```

Table created.

Create Inventory Table

```
SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7),Product_ID VARCHAR(7),Seller_ID VARCHAR(7),Stock_Quantity NUMBER,Stock_Status VARCHAR(11),Last_Updated DATE);
```

Table created.

Primary Key Constraints

Primary keys uniquely identify records in the Seller and Inventory tables.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY (Seller_ID);  
Table altered.  
  
SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inventory PRIMARY KEY (Inventory_ID);  
Table altered.
```

Foreign Key Constraints

Inventory is connected to the Product table from Week 2 using PRODUCT_ID and to the Seller table using SELLER ID.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);  
Table altered.  
  
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);  
Table altered.
```

Unique Email Constraint

The UNIQUE constraint prevents two sellers from having the same email address.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
Table altered.
```

NOT NULL Constraint

The seller email is made mandatory.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
Table altered.
```

CHECK Constraint

The CHECK constraint ensures that stock quantity cannot be negative.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity>=0);  
Table altered.
```

Insert Seller Data

```
INSERT INTO SELLER VALUES('&SELLER_ID','&SELLER_NAME','&EMAIL','&PHONE','&ADDRESS');
```

```
SQL> INSERT INTO Seller VALUES('&Slller_ID','&Seller_Name','&email','&phone','&Address');  
Enter value for selller_id: S01  
Enter value for seller_name: NAUGHTY BITES  
Enter value for email: naughtybites@gmail.com  
Enter value for phone: 9878675645  
Enter value for address: Chrompet  
old 1: INSERT INTO Seller VALUES('&Slller_ID','&Seller_Name','&email','&phone','&Address')  
new 1: INSERT INTO Seller VALUES('S01','NAUGHTY BITES','naughtybites@gmail.com',9878675645,'Chrompet')  
  
1 SQL> /  
S Enter value for selller_id: S05  
E Enter value for seller_name: STAR BIRYANI  
E Enter value for email: starbiryani@gmail.com  
E Enter value for phone: 7654890467  
E Enter value for address: Potheri  
o old 1: INSERT INTO Seller VALUES('&Slller_ID','&Seller_Name','&email','&phone','&Address')  
1 new 1: INSERT INTO Seller VALUES('S05','STAR BIRYANI','starbiryani@gmail.com',7654890467,'Potheri')  
  
S  
E 1 row created.  
E  
Enter value for email: kingofwing@gmail.com  
Enter value for phone: 5874639023  
Enter value for address: Tambaram  
old 1: INSERT INTO Seller VALUES('&Slller_ID','&Seller_Name','&email','&phone','&Address')  
new 1: INSERT INTO Seller VALUES('S03','KING OG WINGS','kingofwing@gmail.com',5874639023,'Tambaram')  
  
1 row created.  
  
SQL> /  
Enter value for selller_id: S04  
Enter value for seller_name: HONEY SPICE  
Enter value for email: honeyspice@gmail.com  
Enter value for phone: 8709675478  
Enter value for address: Urapakkam  
old 1: INSERT INTO Seller VALUES('&Slller_ID','&Seller_Name','&email','&phone','&Address')  
new 1: INSERT INTO Seller VALUES('S04','HONEY SPICE','honeyspice@gmail.com',8709675478,'Urapakkam')  
  
1 row created.
```

Display Seller Records

SELLER_	SELLER_NAME	EMAIL	PHONE	ADDRESS
S01	NAUGHTY BITES	naughtybites@gmail.com	9878675645	Chrompet
S02	MOUNT CAFE	mountcafe@gmail.com	8678584623	St.Thomas mount
S03	KING OG WINGS	kingofwing@gmail.com	5874639023	Tambaram
S04	HONEY SPICE	honeyspice@gmail.com	8709675478	Urapakkam
S05	STAR BIRYANI	starbiryani@gmail.com	7654890467	Potheri

Insert Inventory Data

```
INSERT INTO INVENTORY
VALUES ('&INVID', '&PRODID', '&SELL', &STOCK, '&STOCK_STATUS',
TO_DATE ('&LAST_UPD', 'YYYY-MM-DD'));
```

```
SQL> INSERT INTO INVENTORY VALUES (&invid, &prodid, &sell, &stock, &stock_status, TO_DATE (&last_upd, 'YYYY-MM-DD'));
Enter value for invid: I01
Enter value for prodid: P01
Enter value for sell: S01
Enter value for stock: 50
Enter value for stock_status: Available
Enter value for last_upd: 2026-09-19
old 1: INSERT INTO Inventory VALUES('&InvID', '&ProdID', '&Sell', '&Stock', '&Stock_status', TO_DATE('&Last_upd', 'YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('I01', 'P01', 'S01', 50, 'Available', TO_DATE('2026-09-19', 'YYYY-MM-DD'))
```

1 row created.

```
SQL> /
Enter value for invid: I02
Enter value for prodid: P02
Enter value for sell: S02
Enter value for stock: 30
Enter value for stock_status: Available
Enter value for last_upd: 2026-08-14
old 1: INSERT INTO Inventory VALUES('&InvID', '&ProdID', '&Sell', '&Stock', '&Stock_status', TO_DATE('&Last_upd', 'YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('I02', 'P02', 'S02', 30, 'Available', TO_DATE('2026-08-14', 'YYYY-MM-DD'))
```

1 row created.

```
SQL> /
Enter value for invid: I03
Enter value for prodid: P03
Enter value for sell: S03
Enter value for stock: 25
Enter value for stock_status: Available
Enter value for last_upd: 2026-07-23
old 1: INSERT INTO Inventory VALUES('&InvID', '&ProdID', '&Sell', '&Stock', '&Stock_status', TO_DATE('&Last_upd', 'YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('I03', 'P03', 'S03', 25, 'Available', TO_DATE('2026-07-23', 'YYYY-MM-DD'))
```

1 row created.

```
SQL> /
Enter value for invid: I04
Enter value for prodid: P05
Enter value for sell: S04
Enter value for stock: 35
Enter value for stock_status: Available
Enter value for last_upd: 2026-06-18
old 1: INSERT INTO Inventory VALUES('&InvID', '&ProdID', '&Sell', '&Stock', '&Stock_status', TO_DATE('&Last_upd', 'YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('I04', 'P05', 'S04', 35, 'Available', TO_DATE('2026-06-18', 'YYYY-MM-DD'))
```

```

SQL> /
Enter value for invId: I05
Enter value for prodId: P05
Enter value for sell: S05
Enter value for stock: 78
Enter value for stock_status: Available
Enter value for last_upd: 2026-05-05
old 1: INSERT INTO Inventory VALUES('&InvID','&ProdID','&Sell',&Stock,'&Stock_status',TO_DATE('&Last_upd','YYYY-MM-DD'))
new 1: INSERT INTO Inventory VALUES('I05','P05','S05',78,'Available',TO_DATE('2026-05-05','YYYY-MM-DD'))

1 row created.

```

Display Inventory Records

```

SQL> SELECT * FROM Inventory;

```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATU	LAST_UPDA
I01	P01	S01	50	Available	19-SEP-26
I02	P02	S02	30	Available	14-AUG-26
I03	P03	S03	25	Available	23-JUL-26
I04	P05	S04	35	Available	18-JUN-26
I05	P05	S05	78	Available	05-MAY-26

Seller-wise Available Products and Stock

Purpose: Displays each seller, number of products supplied, and total available stock.

```

SQL> SELECT s.Seller_Name, COUNT(i.Product_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID
= i.Seller_ID WHERE i.Stock_Status = 'Available' GROUP BY s.Seller_Name;

```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
NAUGHTY BITES	1	50
MOUNT CAFE	1	30
STAR BIRYANI	1	78
KING OG WINGS	1	25
HONEY SPICE	1	35

Product Stock and Status

```
SELECT P.PRODUCT_NAME, I.STOCK_QUANTITY, I.STOCK_STATUS
FROM PRODUCT P JOIN INVENTORY I ON P.PRODUCT_ID=I.PRODUCT_ID;
```

Purpose: Displays product name, stock quantity, and stock status.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
IDLI	50	Available
BUTTER NAAN	30	Available
CHICKEN FRIED RICE	25	Available
BROWNIE	35	Available
BROWNIE	78	Available

Count Available Products

```
SELECT COUNT(*) AS TOTAL_AVAILABLE_PRODUCTS
FROM INVENTORY WHERE STOCK_STATUS='Available';
```

Purpose: Counts inventory records whose status is Available.

```
SQL> SELECT COUNT(*) AS Total_Available_Products FROM Inventory WHERE Stock_Status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
5

Count Out-of-Stock Products

```
SELECT COUNT(*) AS OUTOFSTOCK_PRODUCTS
FROM INVENTORY WHERE STOCK_STATUS='Out of Stock';
```

Purpose: Counts inventory records whose status is Out of Stock.

```
SQL> SELECT COUNT(*) AS OutOfStock_Products FROM Inventory WHERE Stock_Status = 'Out of Stock';
```

OUTOFSTOCK_PRODUCTS
0

Product with Maximum Stock

```
SELECT P.PRODUCT_NAME, I.STOCK_QUANTITY
FROM PRODUCT P JOIN INVENTORY I ON P.PRODUCT_ID=I.PRODUCT_ID
WHERE I.STOCK_QUANTITY=(SELECT MAX(STOCK_QUANTITY) FROM INVENTORY);
```

Purpose: Finds the food product having the highest stock quantity.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);
```

```
PRODUCT_NAME      STOCK_QUANTITY
-----
BROWNIE           78
```

20. Average Inventory

```
SELECT AVG(STOCK_QUANTITY) AS AVERAGE_INVENTORY
FROM INVENTORY;
```

Purpose: Calculates the average stock quantity across all inventory records.

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;

AVERAGE_INVENTORY
-----
43.6
```

SQL Concepts Used

SQL Concept	Purpose
CREATE TABLE	Creates Seller and Inventory tables
ALTER TABLE	Adds constraints
PRIMARY KEY	Uniquely identifies records
FOREIGN KEY	Connects Product/Seller with Inventory
UNIQUE	Prevents duplicate seller emails
NOT NULL	Makes seller email mandatory
CHECK	Prevents negative stock
INSERT	Adds records
SELECT	Retrieves records
JOIN	Combines related tables
COUNT()	Counts records
SUM()	Calculates total stock
MAX()	Finds highest stock
AVG()	Calculates average stock
GROUP BY	Groups records

WHERE	Filters records
Subquery	Finds maximum inventory
TO_DATE()	Converts text to DATE

Summary

The Week 3 Food Ordering System implements Seller and Inventory Management using Oracle SQL. The Seller table stores supplier information and the Inventory table stores stock information for food products. Primary keys, foreign keys, UNIQUE, NOT NULL, and CHECK constraints maintain data integrity. Queries using JOIN, COUNT(), SUM(), MAX(), AVG(), GROUP BY, WHERE, and subqueries are used to retrieve and analyze seller and inventory information.